UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

CATEDRÁTICO: ING. JOSE MOISES GRANADOS GUEVARA

ANALISIS DE GRAMATICAS PARA ANALIZADOR LEXICO Y SINTACTICO

PABLO DANIEL ALVARADO RODRIGUEZ

CARNÉ: 202130534

QUETZALTENANGO, MARZO 2026

Análisis De Gramática para Lexer Formularios

Para el análisis de la gramática se tuvo en cuenta la entrada que debe de reconocer el intérprete y a partir de esta se derivó las definiciones de Terminales y No Terminales, concretando este análisis en el analizador Léxico siguiente.

Expresiones Regulares

```
LETRA = [:jletter:]
DIGITO = [0-9]
IDENTIFICADOR = {LETRA}({LETRA}|{DIGITO}|_)*
ENTERO = {DIGITO}+
DECIMAL = {DIGITO}+\.{DIGITO}+
NUMERO = {ENTERO}|{DECIMAL}
COMENTARIO_MULTILINEA = \/\*([^*]|\*[^\/])*\*\/
COMENTARIO_LINEA = \$[^\n\r]*
ESPACIO = [ \t\r\n\f]+
COLOR_HEX = #[0-9A-Fa-f]{6}|#[0-9A-Fa-f]{3}

// --- PALABRAS RESERVADAS ---

    "SECTION"          { return symbol(sym.SECTION); }
    "TABLE"            { return symbol(sym.TABLE); }
    "TEXT"             { return symbol(sym.TEXT); }
    "OPEN_QUESTION"    { return symbol(sym.OPEN_QUESTION); }
    "DROP_QUESTION"    { return symbol(sym.DROP_QUESTION); }
    "SELECT_QUESTION"  { return symbol(sym.SELECT_QUESTION); }
    "MULTIPLE_QUESTION" { return symbol(sym.MULTIPLE_QUESTION); }

    // --- ATRIBUTOS DE FORMULARIO ---

    "width"            { return symbol(sym.WIDTH); }
    "height"           { return symbol(sym.HEIGHT); }
    "pointX"           { return symbol(sym.POINT_X); }
    "pointY"           { return symbol(sym.POINT_Y); }
    "orientation"      { return symbol(sym.ORIENTATION); }
    "elements"         { return symbol(sym.ELEMENTS); }
    "styles"           { return symbol(sym.STYLES); }
    "\"color\""        { return symbol(sym.COLOR_ATTR); }
    "\"background color\"" { return symbol(sym.BACKGROUND_COLOR); }
    "\"font family\""  { return symbol(sym.FONT_FAMILY); }
    "\"text size\""    { return symbol(sym.TEXT_SIZE); }
    "\"border\""       { return symbol(sym.BORDER); }
    "content"          { return symbol(sym.CONTENT); }
    "label"            { return symbol(sym.LABEL); }
    "options"          { return symbol(sym.OPTIONS); }
```

```
"correct"          { return symbol(sym.CORRECT); }

// --- TIPOS DE DATOS Y VALORES RESERVADOS ---

"number"           { return symbol(sym.TYPE_NUMBER); }
"string"           { return symbol(sym.TYPE_STRING); }
"special"          { return symbol(sym.TYPE_SPECIAL); }
"VERTICAL"         { return symbol(sym.VERTICAL); }
"HORIZONTAL"       { return symbol(sym.HORIZONTAL); }
"MONO"             { return symbol(sym.MONO); }
"SANS_SERIF"       { return symbol(sym.SANS_SERIF); }
"CURSIVE"          { return symbol(sym.CURSIVE); }
"LINE"             { return symbol(sym.LINE); }
"DOTTED"           { return symbol(sym.DOTTED); }
"DOUBLE"           { return symbol(sym.DOUBLE); }

// --- PALABRA RESERVADA UTILIZADA PARA LA POKEAPI ---
"NUMBER"           { return symbol(sym.NUMBER); }
// --- COLORES POR DEFECTO ---

"RED" | "BLUE" | "GREEN" | "PURPLE" | "SKY" | "YELLOW" | "BLACK" | "WHITE"
                { return symbol(sym.COLOR_NAME, yytext()); }

// --- CONTROL ---

"IF"               { return symbol(sym.IF); }
"ELSE"             { return symbol(sym.ELSE); }
"WHILE"            { return symbol(sym.WHILE); }
"DO"               { return symbol(sym.DO); }
"FOR"              { return symbol(sym.FOR); }
"in"               { return symbol(sym.IN); }

// --- FUNCIONES ESPECIALES ---

"who_is_that_pokemon" { return symbol(sym.FUNC_POKEMON); }
"draw"             { return symbol(sym.DRAW); }

// --- SÍMBOLOS ESTRUCTURALES ---

"["                { return symbol(sym.CORCHETE_IZQ); }
"]"                { return symbol(sym.CORCHETE_DER); }
"{"                { return symbol(sym.LLAVE_IZQ); }
"}"                { return symbol(sym.LLAVE_DER); }
"("                { return symbol(sym.PAREN_IZQ); }
")"                { return symbol(sym.PAREN_DER); }
","                { return symbol(sym.COMA); }
":"                { return symbol(sym.DOS_PUNTOS); }
";"                { return symbol(sym.PUNTO_COMA); }
```

```
"="             { return symbol(sym.ASIGNACION); }
"?"             { return symbol(sym.COMODIN); }
"."             { return symbol(sym.PUNTO); }

// --- OPERADORES ARITMÉTICOS ---

"+"             { return symbol(sym.MAS); }
"-"             { return symbol(sym.MENOS); }
"*"             { return symbol(sym.POR); }
"/"             { return symbol(sym.DIV); }
"^"             { return symbol(sym.POTENCIA); }
"%"             { return symbol(sym.MODULO); }

// --- OPERADORES RELACIONALES ---

">="            { return symbol(sym.MAYOR_IGUAL); }
"<="            { return symbol(sym.MENOR_IGUAL); }
">"             { return symbol(sym.MAYOR); }
"<"             { return symbol(sym.MENOR); }
"=="            { return symbol(sym.IGUALDAD); }
"!!"            { return symbol(sym.DIFERENTE); }

// --- OPERADORES LÓGICOS ---

"||"            { return symbol(sym.OR); }

"&&"            { return symbol(sym.AND); }
"~"             { return symbol(sym.NOT); }

// --- FORMATOS DE COLOR ---

{COLOR_HEX}     { return symbol(sym.COLOR_HEX, yytext()); }

// --- LITERALES NUMÉRICOS E IDENTIFICADORES ---

{NUMERO}        { return symbol(sym.NUMBER_LITERAL, yytext()); }
{IDENTIFICADOR} { return symbol(sym.IDENTIFICADOR, yytext()); }

// ----- EMOJIS VÁLIDOS -----
// Sonrisa
@\[:\)+\]       { return symbol(sym.EMOJI_SMILE,   yytext()); }
@\[:smile:\]    { return symbol(sym.EMOJI_SMILE,   yytext()); }

// Triste
@\[:\(+\]       { return symbol(sym.EMOJI_SAD,     yytext()); }
@\[:sad:\]      { return symbol(sym.EMOJI_SAD,     yytext()); }

// Serio
```

```
@\[:\|+\]          { return symbol(sym.EMOJI_SERIOUS, yytext()); }
@\[:serious:\]      { return symbol(sym.EMOJI_SERIOUS, yytext()); }

// Corazón
@\[:<+3+\]          { return symbol(sym.EMOJI_HEART,   yytext()); }
@\[:heart:\]        { return symbol(sym.EMOJI_HEART,   yytext()); }

// Gato
@\[:cat:\]          { return symbol(sym.EMOJI_CAT,     yytext()); }
@\[:\^\^:\]        { return symbol(sym.EMOJI_CAT,     yytext()); }

// Estrella estática simple: @[:star:]
@\[:star:\]         { return symbol(sym.EMOJI_STAR, yytext()); }

// Acepta @[:star:5:], @[:star-5:], @[:star+5:]
@\[:star(:|-|\+){DIGITO}+:\] { return symbol(sym.EMOJI_STAR, yytext()); }
```

Análisis De Gramática para Parser Formulario

Para el análisis de la gramática se tuvo en cuenta el análisis previo del Lexer, se tomó en cuenta la estructura de la gramática, las producciones que puede tener el lenguaje y algunos errores que esta puede producir concretando este análisis en el analizador Sintáctico siguiente.

Terminales:
Estos representan las derivaciones que no tienen ninguna otra derivación son estados donde son tokens que ya vienen del análisis Léxico.

```
// Palabras reservadas - Componentes UI
terminal SECTION, TABLE, TEXT, OPEN_QUESTION, DROP_QUESTION;
terminal SELECT_QUESTION, MULTIPLE_QUESTION;

// Atributos de Formulario
terminal WIDTH, HEIGHT, POINT_X, POINT_Y, ORIENTATION, ELEMENTS, STYLES;
terminal COLOR_ATTR, BACKGROUND_COLOR, FONT_FAMILY, TEXT_SIZE, BORDER;
terminal CONTENT, LABEL, OPTIONS, CORRECT;

// Tipos de datos
terminal TYPE_NUMBER, TYPE_STRING, TYPE_SPECIAL;

// Valores de orientación
terminal VERTICAL, HORIZONTAL, MONO, SANS_SERIF, CURSIVE;
```

```
// Tipos de borde
terminal LINE, DOTTED, DOUBLE;

// Colores
terminal String COLOR_NAME, COLOR_HEX, COLOR_RGB, COLOR_HSL;

// Estructuras de control
terminal IF, ELSE, WHILE, DO, FOR, IN;

// Funciones especiales
terminal FUNC_POKEMON, DRAW;

// Símbolos estructurales
terminal CORCHETE_IZQ, CORCHETE_DER, LLAVE_IZQ, LLAVE_DER;
terminal PAREN_IZQ, PAREN_DER, COMA, PUNTO, PUNTO_COMA, DOS_PUNTOS,
ASIGNACION;
terminal COMODIN, NUMBER;

// Operadores aritméticos
terminal MAS, MENOS, POR, DIV, POTENCIA, MODULO;

// Operadores relacionales
terminal MAYOR, MAYOR_IGUAL, MENOR, MENOR_IGUAL, IGUALDAD, DIFERENTE;

// Operadores lógicos
terminal OR, AND, NOT;

// Literales e identificadores
terminal String IDENTIFICADOR, NUMBER_LITERAL;

// Cadenas de texto (manejadas por estado STRING en el lexer)
terminal String INICIO_CADENA, FIN_CADENA, CONTENIDO_CADENA;

// Emojis dentro de cadenas
terminal String EMOJI_SMILE, EMOJI_SAD, EMOJI_SERIOUS, EMOJI_HEART,
EMOJI_CAT, EMOJI_STAR;

// Error léxico
//terminal ERROR;
```

No terminales:

Estos representan estados que generan una lista de derivaciones. Por lo tanto estos permiten la generación de árboles sintácticos para poder analizar la estructura base de aceptación.

```
// Inicio y lista de instrucciones
```

```
non terminal List<NodoInstruccion> inicio;
non terminal List<NodoInstruccion> listaInstrucciones;
non terminal NodoInstruccion instruccion;
non terminal List<NodoInstruccion> listaElementosUI;
non terminal NodoInstruccion elementoUI;

// Expresiones
non terminal NodoExpresion expresion;
non terminal List<NodoExpresion> listaExpresiones;
non terminal List<NodoExpresion> listaExpresionesOpcional;

// Atributos específicos por componente
non terminal List<NodoAtributo> atributosSeccion;
non terminal NodoAtributo atributoSeccion;
non terminal List<NodoAtributo> atributosTabla;
non terminal NodoAtributo atributoTabla;
non terminal List<NodoAtributo> atributosTexto;
non terminal NodoAtributo atributoTexto;
non terminal List<NodoAtributo> atributosPreguntaAbierta;
non terminal NodoAtributo atributoPreguntaAbierta;
non terminal List<NodoAtributo> atributosPreguntaDesplegable;
non terminal NodoAtributo atributoPreguntaDesplegable;
non terminal List<NodoAtributo> atributosSeleccionUnica;
non terminal NodoAtributo atributoSeleccionUnica;
non terminal List<NodoAtributo> atributosMultiple;
non terminal NodoAtributo atributoMultiple;

// Estilos compartidos
non terminal List<NodoAtributo> estilosConBorde;
non terminal NodoAtributo estiloConBorde;
non terminal List<NodoAtributo> estilosSinBorde;
non terminal NodoAtributo estiloSinBorde;

// Declaraciones
non terminal NodoInstruccion declaracion;
non terminal ComponenteUI valorSpecial;

// Sentencias de control
non terminal NodoInstruccion sentenciaIf;
non terminal NodoInstruccion sentenciaWhile;
non terminal NodoInstruccion sentenciaDoWhile;
non terminal NodoInstruccion sentenciaFor;
non terminal NodoInstruccion instruccionForCabecera;
```

```
// Componentes UI
non terminal ComponenteUI componenteSeccion;
non terminal ComponenteUI componenteTexto;
non terminal ComponenteUI componenteTabla;
non terminal ComponenteUI preguntaAbierta;
non terminal ComponenteUI preguntaDesplegable;
non terminal ComponenteUI preguntaSeleccionUnica;
non terminal ComponenteUI preguntaMultiple;

// Instrucciones especiales
non terminal NodoInstruccion instruccionDraw;
non terminal NodoExpresion llamadaAPI;

// Para tablas (filas y celdas)
non terminal List<List<NodoExpresion>> listaFilas;
non terminal List<NodoExpresion> fila;

// Valores especiales
non terminal NodoExpresion valorColor;
non terminal NodoExpresion valorOrientacion;
non terminal NodoExpresion valorFuente;
non terminal NodoExpresion valorTipoBorde;
non terminal NodoExpresion celdaTabla;
non terminal List<Object> listaOpciones;

// Cadenas de texto con soporte de emojis
non terminal String cadenaTexto;
non terminal String partesCadena;
non terminal String parteCadena;
```

Precedencia:
Indica el valor que tiene la mayor precedencia, es decir que mientras más arriba este declarado (antes de la siguiente instrucción), el parser seguirá el camino que este antes y si en dado caso no cumple aún sigue bajando en su árbol sintáctico o más conocido como **AST**.

```
precedence left OR;
precedence left AND;
precedence right NOT;
precedence left IGUALDAD, DIFERENTE;
precedence left MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL;
precedence left MAS, MENOS;
precedence left POR, DIV, MODULO;
precedence right POTENCIA;
```

VISUALIZACION DE LA PRECEDENCIA:

Nivel 1 (más débil) →
|| (OR lógico)

Nivel 2 →
&& (AND lógico)

Nivel 3 →
== != (igualdad / diferencia)

Nivel 4 →
> < >= <= (relacionales)

Nivel 5 →
+ - (suma / resta)

Nivel 6 →
*** / % (multiplicación / división / módulo)**

Nivel 7 →
! (NOT lógico, unario)

Nivel 8 (más fuerte) →
^ (POTENCIA, derecha)

VISUALIZACION DEL ARBOL SINTACTICO:

```
//================================
// Reglas Gramaticales
//================================

start with inicio;

//--- Inicio ---
inicio ::=
    listaInstrucciones:lista
    ;

//--- Lista de Instrucciones ---
listaInstrucciones ::=
    listaInstrucciones:lista instruccion:inst
    | instruccion:inst
    | listaInstrucciones:lista error
    | error
    ;
```

```
//--- Instrucciones ---
instruccion ::=
   // Declaración
   declaracion:d
   | IDENTIFICADOR:id ASIGNACION expresion:exp
   | sentenciaIf:s
   | sentenciaWhile:w
   | sentenciaDoWhile:dw
   | sentenciaFor:f
   | instruccionDraw:d
   | componenteSeccion:c
   | componenteTabla:t
   ;

//--- Lista de elementos UI (solo para SECTION.elements) ---
listaElementosUI ::=
   listaElementosUI:lista COMA elementoUI:elem
   | elementoUI:elem
   | listaElementosUI:lista COMA error
   | error
   ;

elementoUI ::=
   componenteSeccion:c
   | componenteTabla:t
   | componenteTexto:c
   | preguntaAbierta:p
   | preguntaDesplegable:p
   | preguntaSeleccionUnica:p
   | preguntaMultiple:p
   | instruccionDraw:d
   ;

//--- Declaraciones ---
declaracion ::=
   // Declaración sin inicialización
   TYPE_NUMBER:t IDENTIFICADOR:nombre

   | TYPE_STRING:t IDENTIFICADOR:nombre

   // Declaración con inicialización
   | TYPE_NUMBER:t IDENTIFICADOR:nombre ASIGNACION expresion:val

   | TYPE_STRING:t IDENTIFICADOR:nombre ASIGNACION expresion:val


   | TYPE_SPECIAL:t IDENTIFICADOR:nombre ASIGNACION valorSpecial:preg
```

```
    ;

//--- Valor Special (tipos de pregunta que se asignan a variables special) ---
valorSpecial ::=
    preguntaAbierta:p

    | preguntaDesplegable:p

    | preguntaSeleccionUnica:p

    | preguntaMultiple:p
    ;

//--- Sentencia IF ---
sentenciaIf ::=
    // IF sin ELSE
    IF:t PAREN_IZQ expresion:cond PAREN_DER LLAVE_IZQ listaInstrucciones:ifBody
LLAVE_DER

    // IF con ELSE (bloque)
    | IF:t PAREN_IZQ expresion:cond PAREN_DER LLAVE_IZQ
listaInstrucciones:ifBody LLAVE_DER
      ELSE LLAVE_IZQ listaInstrucciones:elseBody LLAVE_DER

    // IF con ELSE IF (encadenado)
    | IF:t PAREN_IZQ expresion:cond PAREN_DER LLAVE_IZQ
listaInstrucciones:ifBody LLAVE_DER
      ELSE sentenciaIf:elseIf
    {:
        // Convertir el siguiente IF en una lista de instrucciones para el ELSE
        List<NodoInstruccion> elseBody = new ArrayList<>();
        elseBody.add(elseIf);
        RESULT = new NodoSentenciaIf(cond, ifBody, elseBody, tleft, tright);
    :}
    ;

//--- Ciclo WHILE ---
sentenciaWhile ::=
    WHILE:t PAREN_IZQ expresion:cond PAREN_DER LLAVE_IZQ
listaInstrucciones:body LLAVE_DER
    ;

//--- Ciclo DO-WHILE ---
sentenciaDoWhile ::=
    DO:t LLAVE_IZQ listaInstrucciones:body LLAVE_DER WHILE PAREN_IZQ
expresion:cond PAREN_DER
    ;
```

```
//--- Ciclo FOR ---
sentenciaFor ::=
   FOR:t PAREN_IZQ IDENTIFICADOR:variable IN expresion:inicio PUNTO PUNTO
expresion:fin PAREN_DER LLAVE_IZQ listaInstrucciones:body LLAVE_DER

   | FOR:t IDENTIFICADOR:variable IN expresion:inicio PUNTO PUNTO expresion:fin
LLAVE_IZQ listaInstrucciones:body LLAVE_DER

   | FOR:t PAREN_IZQ instruccionForCabecera:init PUNTO_COMA expresion:cond
PUNTO_COMA instruccionForCabecera:act PAREN_DER LLAVE_IZQ
listaInstrucciones:body LLAVE_DER
   ;

instruccionForCabecera ::=
   IDENTIFICADOR:nombre ASIGNACION expresion:val
   ;

//--- Instrucción DRAW ---
// Sintaxis: miPregunta.draw(10, 4)  →  IDENTIFICADOR PUNTO DRAW PAREN_IZQ
params PAREN_DER
instruccionDraw ::=
   IDENTIFICADOR:varEspecial PUNTO DRAW:t PAREN_IZQ
listaExpresionesOpcional:params PAREN_DER
   ;

//--- Componente SECTION ---
componenteSeccion ::=
   SECTION:t CORCHETE_IZQ atributosSeccion:attrs CORCHETE_DER
   {:
      List<NodoInstruccion> elementos = new ArrayList<>();
      List<NodoAtributo> attrsFiltrados = new ArrayList<>();
      for (NodoAtributo attr : attrs) {
         if (attr.getNombre().equals("elements")) {
            elementos = (List<NodoInstruccion>) attr.getValor();
         } else {
            attrsFiltrados.add(attr);
         }
      }
      RESULT = new ComponenteSeccion(attrsFiltrados, elementos, tleft, tright);
   :}
   ;

//--- Componente TEXT ---
componenteTexto ::=
   TEXT:t CORCHETE_IZQ atributosTexto:attrs CORCHETE_DER
   {: RESULT = new ComponenteTexto(attrs, tleft, tright); :}
   ;
```

```
//--- Componente TABLE ---
componenteTabla ::=
   TABLE:t CORCHETE_IZQ atributosTabla:attrs CORCHETE_DER
   {:
      List<List<NodoExpresion>> filas = new ArrayList<>();
      List<NodoAtributo> attrsFiltrados = new ArrayList<>();
      for (NodoAtributo attr : attrs) {
         if (attr.getNombre().equals("elements")) {
            filas = (List<List<NodoExpresion>>) attr.getValor();
         } else {
            attrsFiltrados.add(attr);
         }
      }
      RESULT = new ComponenteTabla(attrsFiltrados, filas, tleft, tright);
   :}
   ;

//--- Pregunta Abierta ---
preguntaAbierta ::=
   OPEN_QUESTION:t CORCHETE_IZQ atributosPreguntaAbierta:attrs
CORCHETE_DER
   {: RESULT = new PreguntaAbierta(attrs, tleft, tright); :}
   ;

//--- Pregunta Desplegable ---
preguntaDesplegable ::=
   DROP_QUESTION:t CORCHETE_IZQ atributosPreguntaDesplegable:attrs
CORCHETE_DER
   {: RESULT = new PreguntaDesplegable(attrs, tleft, tright); :}
   ;

//--- Pregunta Selección Única ---
preguntaSeleccionUnica ::=
   SELECT_QUESTION:t CORCHETE_IZQ atributosSeleccionUnica:attrs
CORCHETE_DER
   {: RESULT = new PreguntaSeleccionUnica(attrs, tleft, tright); :}
   ;

//--- Pregunta Selección Múltiple ---
preguntaMultiple ::=
   MULTIPLE_QUESTION:t CORCHETE_IZQ atributosMultiple:attrs CORCHETE_DER
   ;

//--- Expresiones ---
expresion ::=
   // Operaciones aritméticas
   expresion:izq MAS:op expresion:der
```

| expresion:izq MENOS:op expresion:der

| expresion:izq POR:op expresion:der

| expresion:izq DIV:op expresion:der

| expresion:izq POTENCIA:op expresion:der

| expresion:izq MODULO:op expresion:der

// Operaciones relacionales
| expresion:izq MAYOR:op expresion:der

| expresion:izq MENOR:op expresion:der

| expresion:izq MAYOR_IGUAL:op expresion:der

| expresion:izq MENOR_IGUAL:op expresion:der

| expresion:izq IGUALDAD:op expresion:der

| expresion:izq DIFERENTE:op expresion:der

// Operaciones lógicas
| expresion:izq OR:op expresion:der

| expresion:izq AND:op expresion:der

| NOT:op expresion:exp

// Menos unario
| MENOS:op expresion:exp

// Literales
| NUMBER_LITERAL:num

| cadenaTexto:str

// Valores especiales
| valorColor:c

| valorOrientacion:o

// Acceso a variable
| IDENTIFICADOR:id

// Llamada a API
| llamadaAPI:api

```
    // Agrupación
    | PAREN_IZQ expresion:exp PAREN_DER

    // Comodín
    | COMODIN:t
    ;

//--- Llamada a API (who_is_that_pokemon) ---
llamadaAPI ::=
    FUNC_POKEMON:t PAREN_IZQ NUMBER COMA expresion:inicio COMA
expresion:fin PAREN_DER
    ;

//--- Lista de Expresiones (para draw y otros) ---
listaExpresionesOpcional ::=
    listaExpresiones:lista
    |
    | error
    ;

listaExpresiones ::=
    listaExpresiones:lista COMA expresion:exp
    {:
        lista.add(exp);
    :}
    | expresion:exp
    {:
        List<NodoExpresion> lista = new ArrayList<NodoExpresion>();
        lista.add(exp);
    :}
    ;

//--- Atributos para SECTION ---
// SECTION admite: width, height, pointX, pointY, orientation(opt), elements(opt),
styles(opt)
atributosSeccion ::=
    atributosSeccion:list COMA atributoSeccion:a
    {: list.add(a); RESULT = list; :}
    | atributoSeccion:a
    {:  List<NodoAtributo> l = new ArrayList<NodoAtributo>(); l.add(a); RESULT = l; :}
    | atributosSeccion:list COMA error
    | error
    ;

atributoSeccion ::=
    WIDTH DOS_PUNTOS expresion:v
```

```
    | HEIGHT DOS_PUNTOS expresion:v

    | POINT_X DOS_PUNTOS expresion:v

    | POINT_Y DOS_PUNTOS expresion:v

    | ORIENTATION DOS_PUNTOS valorOrientacion:v

    | ELEMENTS DOS_PUNTOS LLAVE_IZQ listaElementosUI:elems LLAVE_DER

    | STYLES CORCHETE_IZQ estilosConBorde:ests CORCHETE_DER
    //* Agregar la produccion de draw
    ;

//--- Atributos para TABLE ---
// TABLE admite: width, height, pointX, pointY, elements(filas), styles(opt)
atributosTabla ::=
    atributosTabla:list COMA atributoTabla:a
    | atributoTabla:a
    | atributosTabla:list COMA error
    | error
    ;

atributoTabla ::=
    WIDTH DOS_PUNTOS expresion:v

    | HEIGHT DOS_PUNTOS expresion:v

    | POINT_X DOS_PUNTOS expresion:v

    | POINT_Y DOS_PUNTOS expresion:v

    | ELEMENTS DOS_PUNTOS LLAVE_IZQ listaFilas:filas LLAVE_DER

    | STYLES CORCHETE_IZQ estilosConBorde:ests CORCHETE_DER
    ;

//--- Atributos para TEXT ---
// TEXT admite: width(opt), height(opt), content, styles(opt) — sin
pointX/pointY/border
atributosTexto ::=
    atributosTexto:list COMA atributoTexto:a
    | atributoTexto:a
    | atributosTexto:list COMA error
    | error
    ;

atributoTexto ::=
```

```
    WIDTH DOS_PUNTOS expresion:v

    | HEIGHT DOS_PUNTOS expresion:v

    | CONTENT DOS_PUNTOS expresion:v

    | STYLES CORCHETE_IZQ estilosSinBorde:ests CORCHETE_DER
    ;

//--- Atributos para OPEN_QUESTION ---
// OPEN_QUESTION admite: width(opt), height(opt), label — sin
pointX/pointY/options/correct/border
atributosPreguntaAbierta ::=
    atributosPreguntaAbierta:list COMA atributoPreguntaAbierta:a
    | atributoPreguntaAbierta:a
    | atributosPreguntaAbierta:list COMA error
    {: RESULT = list; :}
    | error
    ;

atributoPreguntaAbierta ::=
    WIDTH DOS_PUNTOS expresion:v

    | HEIGHT DOS_PUNTOS expresion:v

    | LABEL DOS_PUNTOS expresion:v

    | STYLES CORCHETE_IZQ estilosSinBorde:ests CORCHETE_DER
    ;

//--- Atributos para DROP_QUESTION ---
// DROP_QUESTION admite: width(opt), height(opt), label, options, correct(opt),
styles(opt)
atributosPreguntaDesplegable ::=
    atributosPreguntaDesplegable:list COMA atributoPreguntaDesplegable:a
    | atributoPreguntaDesplegable:a
    | atributosPreguntaDesplegable:list COMA error
    | error
    ;

atributoPreguntaDesplegable ::=
    WIDTH DOS_PUNTOS expresion:v

    | HEIGHT DOS_PUNTOS expresion:v

    | LABEL DOS_PUNTOS expresion:v

    | OPTIONS DOS_PUNTOS LLAVE_IZQ listaOpciones:opts LLAVE_DER
```

```
    // options: who_is_that_pokemon(NUMBER, 1, 10)
    | OPTIONS DOS_PUNTOS FUNC_POKEMON:t PAREN_IZQ NUMBER COMA
expresion:inicio COMA expresion:fin PAREN_DER


    // Variante sin options para who_is_that_pokemon(NUMBER, 1, 10)
    | FUNC_POKEMON:t PAREN_IZQ NUMBER COMA expresion:inicio COMA
expresion:fin PAREN_DER

    | CORRECT DOS_PUNTOS expresion:v

    | STYLES CORCHETE_IZQ estilosSinBorde:ests CORCHETE_DER
    ;

//--- Atributos para SELECT_QUESTION ---
// SELECT_QUESTION admite: width(opt), height(opt), label(opt), options, correct(opt),
styles(opt)
atributosSeleccionUnica ::=
    atributosSeleccionUnica:list COMA atributoSeleccionUnica:a
    | atributoSeleccionUnica:a
    | atributosSeleccionUnica:list COMA error
    | error
    ;

atributoSeleccionUnica ::=
    WIDTH DOS_PUNTOS expresion:v

    | HEIGHT DOS_PUNTOS expresion:v

    | LABEL DOS_PUNTOS expresion:v

    | OPTIONS DOS_PUNTOS LLAVE_IZQ listaOpciones:opts LLAVE_DER

    // options: who_is_that_pokemon(NUMBER, 1, 10)
    | OPTIONS DOS_PUNTOS FUNC_POKEMON:t PAREN_IZQ NUMBER COMA
expresion:inicio COMA expresion:fin PAREN_DER

    // Variante de who_is_that_pokemon(NUMBER, 1, 10)
    | FUNC_POKEMON:t PAREN_IZQ NUMBER COMA expresion:inicio COMA
expresion:fin PAREN_DER

    | CORRECT DOS_PUNTOS expresion:v

    | STYLES CORCHETE_IZQ estilosSinBorde:ests CORCHETE_DER
    ;

//--- Atributos para MULTIPLE_QUESTION ---
```

```
// MULTIPLE_QUESTION admite: width(opt), height(opt), label(opt), options,
correct:{n,n}(opt), styles(opt)
atributosMultiple ::=
   atributosMultiple:list COMA atributoMultiple:a
   | atributoMultiple:a
   | atributosMultiple:list COMA error
   | error
   ;

atributoMultiple ::=
   WIDTH DOS_PUNTOS expresion:v

   | HEIGHT DOS_PUNTOS expresion:v

   | LABEL DOS_PUNTOS expresion:v

   | OPTIONS DOS_PUNTOS LLAVE_IZQ listaOpciones:opts LLAVE_DER

   // options: who_is_that_pokemon(NUMBER, 1, 10)
   | OPTIONS DOS_PUNTOS FUNC_POKEMON:t PAREN_IZQ NUMBER COMA
expresion:inicio COMA expresion:fin PAREN_DER


   // Variante who_is_that_pokemon(NUMBER, 1, 10)
   | FUNC_POKEMON:t PAREN_IZQ NUMBER COMA expresion:inicio COMA
expresion:fin PAREN_DER


   // correct en MULTIPLE_QUESTION es una lista de índices: correct: {1, 3}
   | CORRECT DOS_PUNTOS LLAVE_IZQ listaExpresiones:corrs LLAVE_DER

   | STYLES CORCHETE_IZQ estilosSinBorde:ests CORCHETE_DER
   ;

//--- Estilos con borde (para SECTION y TABLE) ---
estilosConBorde ::=
   estilosConBorde:list COMA estiloConBorde:e
   | estiloConBorde:e
   ;

estiloConBorde ::=
   COLOR_ATTR DOS_PUNTOS valorColor:v

   | BACKGROUND_COLOR DOS_PUNTOS valorColor:v

   | FONT_FAMILY DOS_PUNTOS valorFuente:v

   | TEXT_SIZE DOS_PUNTOS expresion:v
```

```
   | BORDER DOS_PUNTOS PAREN_IZQ expresion:grosor COMA valorTipoBorde:tipo
COMA valorColor:color PAREN_DER


   | error
   {: RESULT = null; :}
   ;

//--- Estilos sin borde (para TEXT y preguntas) ---
estilosSinBorde ::=
   estilosSinBorde:list COMA estiloSinBorde:e
   | estiloSinBorde:e
   ;

estiloSinBorde ::=
   COLOR_ATTR DOS_PUNTOS valorColor:v

   | BACKGROUND_COLOR DOS_PUNTOS valorColor:v

   | FONT_FAMILY DOS_PUNTOS valorFuente:v

   | TEXT_SIZE DOS_PUNTOS expresion:v

   | error
   ;

//--- Filas de Tabla ---
listaFilas ::=
   listaFilas:lista COMA CORCHETE_IZQ fila:f CORCHETE_DER

   | CORCHETE_IZQ fila:f CORCHETE_DER

   ;

fila ::=
   fila:lista COMA celdaTabla:exp

   | celdaTabla:exp
   ;

celdaTabla ::=
   LLAVE_IZQ:t elementoUI:elem LLAVE_DER

   | LLAVE_IZQ expresion:exp LLAVE_DER
   | expresion:exp
   ;
```

```
//--- Lista de Opciones (para preguntas) ---
listaOpciones ::=
   listaOpciones:lista COMA expresion:opcion

   | expresion:opcion

   | listaOpciones:lista COMA error
   | error
   ;

//--- Valor de Color ---
valorColor ::=
   COLOR_NAME:nombre

   | COLOR_HEX:hex

   | COLOR_RGB:rgb

   | COLOR_HSL:hsl

   // Permiten colores con expresiones: (expr, expr, expr) o <expr, expr, expr>
   | PAREN_IZQ expresion:r COMA expresion:g COMA expresion:b PAREN_DER


   | MENOR expresion:h COMA expresion:s COMA expresion:l MAYOR

   ;

//--- Valor de Orientación ---
valorOrientacion ::=
   VERTICAL:t

   | HORIZONTAL:t
   ;

valorFuente ::=
   MONO:t

   | SANS_SERIF:t

   | CURSIVE:t
   ;

valorTipoBorde ::=
   LINE:t

   | DOTTED:t
```

```
    | DOUBLE:t
    ;

//--- Cadena de texto ---
// Una cadena puede estar vacía o tener una o más partes
cadenaTexto ::=
    INICIO_CADENA partesCadena:partes FIN_CADENA
    | INICIO_CADENA FIN_CADENA
    ;

// Acumulación de partes (texto + tokens especiales)
partesCadena ::=
    partesCadena:acum parteCadena:parte
    | parteCadena:parte
    ;

// Cada parte puede ser contenido normal o token especial.
// El parser no transforma emojis: solo conserva el texto fuente.
parteCadena ::=
    CONTENIDO_CADENA:c
    | EMOJI_SMILE:e
    | EMOJI_SAD:e
    | EMOJI_SERIOUS:e
    | EMOJI_HEART:e
    | EMOJI_CAT:e
    | EMOJI_STAR:e
    ;
```

Análisis De Gramática para Lexer Formularios

Para el análisis de la gramática se tuvo en cuenta la entrada que debe de reconocer el intérprete y a partir de esta se derivó las definiciones de Terminales y No Terminales, concretando este análisis en el analizador Léxico siguiente.

Expresiones Regulares

```
LETRA = [:jletter:]
DIGITO = [0-9]
MARCA_COMBINADA = [\u0300-\u036F]
IDENT = {LETRA}({LETRA}|{DIGITO}|{MARCA_COMBINADA}|_|-)*
ENTERO = {DIGITO}+
NUMERO = -?{ENTERO}(\.{ENTERO})?
ESPACIO = [ \t\r\n\f]+
COLOR_HEX_RE = #[0-9A-Fa-f]{3}([0-9A-Fa-f]{3})?
COMP_COLOR = ({NUMERO}|\?)
COLOR_RGB_RE = \({COMP_COLOR}[ ]*,[ ]*{COMP_COLOR}[ ]*,[ ]*{COMP_COLOR}\)
COLOR_HSL_RE = \<{COMP_COLOR}[ ]*,[ ]*{COMP_COLOR}[ ]*,[ ]*{COMP_COLOR}\>
K_TOTAL_SECCIONES = "total"[ ]+"de"[ ]+"secciones"
K_TOTAL_PREGUNTAS = "total"[ ]+"de"[ ]+"preguntas"
K_BACKGROUND_COLOR = "background"[ ]+"color"
K_FONT_FAMILY = "font"[ ]+"family"
K_TEXT_SIZE = "text"[ ]+"size"
```

Palabras Reservadas

```
"###" { yybegin(METADATA); return symbol(sym.METADATA_DELIM); }
"Author" { return symbol(sym.AUTHOR); }
"Fecha" { return symbol(sym.FECHA); }
"Hora" { return symbol(sym.HORA); }
"Description" { return symbol(sym.DESCRIPTION); }
{K_TOTAL_SECCIONES} { return symbol(sym.TOTAL_SECCIONES); }
{K_TOTAL_PREGUNTAS} { return symbol(sym.TOTAL_PREGUNTAS); }
"Abiertas" { return symbol(sym.ABIERTAS); }
"Desplegables" { return symbol(sym.DESPLEGABLES); }
"Selección" { return symbol(sym.SELECCION); }
"Múltiples" { return symbol(sym.MULTIPLES); }
"Total de Componentes" { return symbol(sym.TOTAL_COMPONENTES); }
"Textos"            { return symbol(sym.TEXTOS); }
"Tablas"            { return symbol(sym.TABLAS); }
"Con estilos"         { return symbol(sym.CON_ESTILOS); }
"Draws ejecutados"     { return symbol(sym.DRAWS_EJECUTADOS); }
"section" { return symbol(sym.SECTION); }
"style" { return symbol(sym.STYLE); }
"content" { return symbol(sym.CONTENT); }
"table" { return symbol(sym.TABLE); }
"line" { return symbol(sym.LINE); }
"element" { return symbol(sym.ELEMENT); }
```

```
"open" { return symbol(sym.OPEN); }
"drop" { return symbol(sym.DROP); }
"select" { return symbol(sym.SELECT); }
"multiple" { return symbol(sym.MULTIPLE); }
"color" { return symbol(sym.COLOR_KEY); }
{K_BACKGROUND_COLOR} { return symbol(sym.BACKGROUND_COLOR_KEY); }
{K_FONT_FAMILY} { return symbol(sym.FONT_FAMILY_KEY); }
{K_TEXT_SIZE} { return symbol(sym.TEXT_SIZE_KEY); }
"border" { return symbol(sym.BORDER_KEY); }
"VERTICAL" { return symbol(sym.VERTICAL); }
"HORIZONTAL" { return symbol(sym.HORIZONTAL); }
"MONO" { return symbol(sym.MONO); }
"SANS_SERIF" { return symbol(sym.SANS_SERIF); }
"CURSIVE" { return symbol(sym.CURSIVE); }
"LINE" | "DOTTED" | "DOUBLE" { return symbol(sym.BORDER_STYLE, yytext()); }
"true" | "false" { return symbol(sym.BOOLEAN, yytext()); }
{COLOR_HEX_RE} { return symbol(sym.COLOR_HEX, yytext()); }
{COLOR_RGB_RE} { return symbol(sym.COLOR_RGB, yytext()); }
{COLOR_HSL_RE} { return symbol(sym.COLOR_HSL, yytext()); }
"</" { return symbol(sym.LT_SLASH); }
"/>" { return symbol(sym.SLASH_GT); }
"/" { return symbol(sym.SLASH); }
"<" { return symbol(sym.LT); }
">" { return symbol(sym.GT); }
"=" { return symbol(sym.EQUALS); }
"," { return symbol(sym.COMMA); }
":" { return symbol(sym.COLON); }
"{" { return symbol(sym.LBRACE); }
"}" { return symbol(sym.RBRACE); }
{NUMERO} { return symbol(sym.NUMBER, yytext()); }
{IDENT} { return symbol(sym.IDENTIFIER, yytext()); }
```

Análisis De Gramática para Parser PKM

Para el análisis de la gramática se tuvo en cuenta el análisis previo del Lexer, se tomó en cuenta la estructura de la gramática, las producciones que puede tener el lenguaje y algunos errores que esta puede producir concretando este análisis en el analizador Sintáctico siguiente.

Terminales:
Estos representan las derivaciones que no tienen ninguna otra derivación son estados donde son tokens que ya vienen del análisis Léxico.

```
terminal METADATA_DELIM, AUTHOR, FECHA, HORA, DESCRIPTION;
terminal TOTAL_SECCIONES, TOTAL_PREGUNTAS, ABIERTAS, DESPLEGABLES,
SELECCION, MULTIPLES;
```

```
terminal TOTAL_COMPONENTES, TEXTOS, TABLAS, CON_ESTILOS,
DRAWS_EJECUTADOS;

terminal SECTION, STYLE, CONTENT, TABLE, LINE, ELEMENT, OPEN, DROP, SELECT,
MULTIPLE;
terminal COLOR_KEY, BACKGROUND_COLOR_KEY, FONT_FAMILY_KEY,
TEXT_SIZE_KEY, BORDER_KEY;
terminal VERTICAL, HORIZONTAL, MONO, SANS_SERIF, CURSIVE;
terminal String BORDER_STYLE, BOOLEAN;
terminal String COLOR_HEX, COLOR_RGB, COLOR_HSL, NUMBER, IDENTIFIER;
terminal LT, GT, LT_SLASH, SLASH_GT, EQUALS, COMMA, COLON, SLASH, LBRACE,
RBRACE;
terminal String STRING_START, STRING_END, STRING_TEXT, EMOJI_SPEC;
```

No terminales:
Estos representan estados que generan una lista de derivaciones. Por lo tanto estos permiten la generación de árboles sintácticos para poder analizar la estructura base de aceptación.

```
non terminal Formulario inicio;
non terminal List bloques_no_vacios, bloque;
non terminal ElementoFormulario section_block, section_item;
non terminal List section_body_opt;
non terminal EstiloElemento style_block, style_opt;
non terminal Map style_items_opt, style_item;
non terminal String style_key;
non terminal List content_block, content_items_opt;
non terminal ElementoFormulario content_item;
non terminal ElementoFormulario table_block;
non terminal List line_blocks_no_vacios, line_block, element_blocks_no_vacios;
non terminal ElementoFormulario element_block, element_content_opt;
non terminal ElementoFormulario open_block, drop_block, select_block, multiple_block;
// table_rest representa el contenido interno de <table> despues de consumir "<table".
// Se separa en otro no terminal para evitar conflictos entre tablas con y sin estilo.
non terminal ElementoFormulario table_rest;
non terminal Orientacion orientacion;
non terminal Object valor_simple;
non terminal List lista_valores_opt, lista_valores;
non terminal Object metadata_block, metadata_valor;
non terminal List lista_strings, lista_cadenas, lista_enteros_opt, lista_enteros;
non terminal String cadena, cadena_partes_opt, cadena_parte;
```

Precedencia:
Indica el valor que tiene la mayor precedencia, es decir que mientras más arriba este declarado (antes de la siguiente instrucción), el parser seguirá el camino que este antes y si en dado caso no cumple aún sigue bajando en su árbol sintáctico o más conocido como AST.

VISUALIZACION DEL ARBOL SINTACTICO:

```
start with inicio;

// Punto de entrada del parser PKM:
// primero exige metadatos y luego una lista de bloques renderizables.
inicio ::=
   metadata_block bloques_no_vacios:bloques
   {: RESULT = new Formulario(bloques); :}
   ;

// Bloque de metadatos obligatorio al inicio del archivo.
// Se parsea para validar formato, pero no se agrega al Formulario final.
metadata_block ::=
   METADATA_DELIM
   AUTHOR COLON metadata_valor
   FECHA COLON metadata_valor
   HORA COLON metadata_valor
   DESCRIPTION COLON metadata_valor
   TOTAL_SECCIONES COLON NUMBER
   TOTAL_PREGUNTAS COLON NUMBER
   ABIERTAS COLON NUMBER
   DESPLEGABLES COLON NUMBER
   SELECCION COLON NUMBER
   MULTIPLES COLON NUMBER
   TOTAL_COMPONENTES COLON NUMBER
   TEXTOS COLON NUMBER
   TABLAS COLON NUMBER
   CON_ESTILOS COLON NUMBER
   DRAWS_EJECUTADOS COLON NUMBER
   METADATA_DELIM
   {: RESULT = null; :}
   ;

metadata_valor ::=
   cadena:c {: RESULT = c; :}
   | IDENTIFIER:i {: RESULT = i; :}
   | NUMBER:n {: RESULT = n; :}
   | NUMBER:d1 SLASH NUMBER:d2 SLASH NUMBER:d3 {: RESULT = d1 + "/" + d2 +
"/" + d3; :}
   | NUMBER:h COLON NUMBER:m {: RESULT = h + ":" + m; :}
   | metadata_valor:base IDENTIFIER:i {: RESULT = String.valueOf(base) + " " + i; :}
   | metadata_valor:base NUMBER:n {: RESULT = String.valueOf(base) + " " + n; :}
   ;
```

```
// Concatena bloques top-level y mantiene solo elementos de UI validos.
bloques_no_vacios ::=
   bloque:b
   {: RESULT = b; :}
   | bloques_no_vacios:lista bloque:b
   {: lista.addAll(b); RESULT = lista; :}
   ;

// Cada bloque top-level devuelve una lista para simplificar concatenacion.
// style_block suelto no genera nodo visual directo, por eso regresa lista vacia.
bloque ::=
   section_block:s
   {: RESULT = experto.listaConElementoSiNoNulo(s); :}
   | style_block
   {: RESULT = experto.listaVacia(); :}
   | content_block:c
   {: RESULT = c; :}
   | table_block:t
   {: RESULT = experto.listaConElementoSiNoNulo(t); :}
   | open_block:o
   {: RESULT = experto.listaConElementoSiNoNulo(o); :}
   | drop_block:d
   {: RESULT = experto.listaConElementoSiNoNulo(d); :}
   | select_block:s
   {: RESULT = experto.listaConElementoSiNoNulo(s); :}
   | multiple_block:m
   {: RESULT = experto.listaConElementoSiNoNulo(m); :}
   ;

// Seccion con geometria (w,h,x,y), orientacion y contenido interno.
section_block ::=
   LT SECTION EQUALS valor_simple:w COMMA valor_simple:h COMMA
valor_simple:x COMMA valor_simple:y COMMA orientacion:o GT
   section_body_opt:hijos
   LT_SLASH SECTION GT
   {: RESULT = new SeccionFormulario(
         Float.valueOf(experto.toFloat(w)),
         Float.valueOf(experto.toFloat(h)),
         experto.toFloat(x),
         experto.toFloat(y),
         o,
         hijos,
         experto.estiloDefault()
      );
   :}
   ;
```

```
section_body_opt ::=
   section_body_opt:lista section_item:item
   {: if (item != null) { lista.add(item); } RESULT = lista; :}
   | section_body_opt:lista content_block:items
   {: if (items != null) { lista.addAll(items); } RESULT = lista; :}
   | /* vacio */
   {: RESULT = experto.listaVacia(); :}
   ;

// Dentro de section pueden venir items y tambien style suelto.
// style suelto se ignora aqui porque no hay asignacion de estilo a nivel section aun.
section_item ::=
   style_block
   {: RESULT = null; :}
   | content_item:c
   {: RESULT = c; :}
   ;

orientacion ::=
   VERTICAL
   {: RESULT = Orientacion.VERTICAL; :}
   | HORIZONTAL
   {: RESULT = Orientacion.HORIZONTAL; :}
   ;

// <style> ... </style> se transforma a EstiloElemento via el experto.
style_block ::=
   LT STYLE GT style_items_opt:items LT_SLASH STYLE GT
   {: RESULT = experto.buildStyleFromMap(items); :}
   ;

// Atributos de style acumulados como mapa clave-valor.
style_items_opt ::=
   style_items_opt:acum style_item:item
   {: acum.putAll(item); RESULT = acum; :}
   | /* vacio */
   {: RESULT = experto.mapaVacio(); :}
   ;

style_item ::=
   LT style_key:k EQUALS valor_simple:v SLASH_GT
   {: RESULT = experto.mapaAtributo(k, v); :}
   | LT BORDER_KEY COMMA valor_simple:w COMMA valor_simple:t COMMA
COLOR_KEY EQUALS valor_simple:c SLASH_GT
   {: RESULT = experto.mapaBorde(w, t, c); :}
   ;

style_key ::=
```

```
    COLOR_KEY
    {: RESULT = "color"; :}
    | BACKGROUND_COLOR_KEY
    {: RESULT = "background_color"; :}
    | FONT_FAMILY_KEY
    {: RESULT = "font_family"; :}
    | TEXT_SIZE_KEY
    {: RESULT = "text_size"; :}
    | BORDER_KEY
    {: RESULT = "border"; :}
    ;

content_block ::=
    LT CONTENT GT content_items_opt:items LT_SLASH CONTENT GT
    {: RESULT = items; :}
    ;

content_items_opt ::=
    content_items_opt:lista content_item:item
    {: if (item != null) { lista.add(item); } RESULT = lista; :}
    | /* vacio */
    {: RESULT = experto.listaVacia(); :}
    ;

// Puede contener preguntas y tambien tablas anidadas.
content_item ::=
    open_block:o
    {: RESULT = o; :}
    | drop_block:d
    {: RESULT = d; :}
    | select_block:s
    {: RESULT = s; :}
    | multiple_block:m
    {: RESULT = m; :}
    | table_block:t
    {: RESULT = t; :}
    | section_block:s
    {: RESULT = s; :}
    ;

// table_block consume la cabecera <table> y delega el resto a table_rest.
// Esto deja explicito donde se decide: tabla con estilo interno o sin estilo.
table_block ::=
    LT TABLE GT
    LT table_rest:t
    {: RESULT = t; :}
    ;
```

```
// table_rest modela lo que puede ir dentro de <table>:
// STYLE + CONTENT + cierre de table
// CONTENT directo + cierre de table (usa estilo por defecto)
// Se usa un no terminal separado para eliminar ambiguedad en CUP.
table_rest ::=
   STYLE GT style_items_opt:items LT_SLASH STYLE GT
   LT CONTENT GT line_blocks_no_vacios:filas LT_SLASH CONTENT GT
   LT_SLASH TABLE GT
   {: RESULT = new TablaFormulario(null, null, 0f, 0f, filas,
experto.buildStyleFromMap(items)); :}
   | CONTENT GT line_blocks_no_vacios:filas LT_SLASH CONTENT GT
   LT_SLASH TABLE GT
   {: RESULT = new TablaFormulario(null, null, 0f, 0f, filas, experto.estiloDefault()); :}
   ;

line_blocks_no_vacios ::=
   line_block:fila
   {: List filas = experto.listaVacia(); filas.add(fila); RESULT = filas; :}
   | line_blocks_no_vacios:filas line_block:fila
   {: filas.add(fila); RESULT = filas; :}
   ;

// Cada <line> produce una fila (lista de elementos).
line_block ::=
   LT LINE GT element_blocks_no_vacios:elementos LT_SLASH LINE GT
   {: RESULT = elementos; :}
   ;

element_blocks_no_vacios ::=
   element_block:e
   {: List elementos = experto.listaVacia(); if (e != null) { elementos.add(e); } RESULT =
elementos; :}
   | element_blocks_no_vacios:lista element_block:e
   {: if (e != null) { lista.add(e); } RESULT = lista; :}
   ;

// Cada <element> encapsula un item de contenido.
// Si viene vacio, se crea un TextoFormulario vacio como placeholder visual.
element_block ::=
   LT ELEMENT GT element_content_opt:contenido LT_SLASH ELEMENT GT
   {: RESULT = (contenido != null)
      ? contenido
      : new TextoFormulario(null, null, "", experto.estiloDefault());
   :}
   ;

element_content_opt ::=
   content_item:c
```

```
   {: RESULT = c; :}
   | valor_simple:v
   {: RESULT = new TextoFormulario(null, null, experto.procesarTexto(v),
experto.estiloDefault()); :}
   | /* vacio */
   {: RESULT = null; :}
   ;

open_block ::=
   LT OPEN EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
SLASH_GT
   {: RESULT = new PreguntaAbierta(
         Float.valueOf(experto.toFloat(w)),
         Float.valueOf(experto.toFloat(h)),
         experto.procesarTexto(label),
         experto.estiloDefault(),
         ""
      );
   :}
   | LT OPEN EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
GT style_opt:s LT_SLASH OPEN GT
   {: RESULT = new PreguntaAbierta(
         Float.valueOf(experto.toFloat(w)),
         Float.valueOf(experto.toFloat(h)),
         experto.procesarTexto(label),
         s,
         ""
      );
   :}
   ;

drop_block ::=
   LT DROP EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
COMMA lista_strings:opts COMMA NUMBER:correcta SLASH_GT
   {: RESULT = new PreguntaDesplegable(
         Float.valueOf(experto.toFloat(w)),
         Float.valueOf(experto.toFloat(h)),
         experto.procesarTexto(label),
         opts,
         experto.normalizarIndice(experto.toInt(correcta), opts, "drop.correcta"),
         experto.estiloDefault(),
         -1
      );
   :}
   | LT DROP EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
COMMA lista_strings:opts COMMA NUMBER:correcta GT style_opt:s LT_SLASH
DROP GT
   {: RESULT = new PreguntaDesplegable(
```

```
            Float.valueOf(experto.toFloat(w)),
            Float.valueOf(experto.toFloat(h)),
            experto.procesarTexto(label),
            opts,
            experto.normalizarIndice(experto.toInt(correcta), opts, "drop.correcta"),
            s,
            -1
        );
    :}
    ;

select_block ::=
    LT SELECT EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
COMMA lista_strings:opts COMMA NUMBER:correcta SLASH_GT
    {: RESULT = new PreguntaSeleccionUnica(
            Float.valueOf(experto.toFloat(w)),
            Float.valueOf(experto.toFloat(h)),
            experto.procesarTexto(label),
            opts,
            experto.normalizarIndice(experto.toInt(correcta), opts, "select.correcta"),
            experto.estiloDefault(),
            -1
        );
    :}
    | LT SELECT EQUALS valor_simple:w COMMA valor_simple:h COMMA cadena:label
COMMA lista_strings:opts COMMA NUMBER:correcta GT style_opt:s LT_SLASH
SELECT GT
    {: RESULT = new PreguntaSeleccionUnica(
            Float.valueOf(experto.toFloat(w)),
            Float.valueOf(experto.toFloat(h)),
            experto.procesarTexto(label),
            opts,
            experto.normalizarIndice(experto.toInt(correcta), opts, "select.correcta"),
            s,
            -1
        );
    :}
    ;

multiple_block ::=
    LT MULTIPLE EQUALS valor_simple:w COMMA valor_simple:h COMMA
cadena:label COMMA lista_strings:opts COMMA LBRACE lista_enteros_opt:correctas
RBRACE SLASH_GT
    {: RESULT = new PreguntaSeleccionMultiple(
            Float.valueOf(experto.toFloat(w)),
            Float.valueOf(experto.toFloat(h)),
            experto.procesarTexto(label),
            opts,
```

```
            experto.filtrarIndicesValidos(correctas, opts, "multiple.correctas"),
            experto.estiloDefault(),
            new HashSet()
        );
    :}
    | LT MULTIPLE EQUALS valor_simple:w COMMA valor_simple:h COMMA
cadena:label COMMA lista_strings:opts COMMA LBRACE lista_enteros_opt:correctas
RBRACE GT style_opt:s LT_SLASH MULTIPLE GT
    {: RESULT = new PreguntaSeleccionMultiple(
            Float.valueOf(experto.toFloat(w)),
            Float.valueOf(experto.toFloat(h)),
            experto.procesarTexto(label),
            opts,
            experto.filtrarIndicesValidos(correctas, opts, "multiple.correctas"),
            s,
            new HashSet()
        );
    :}
    ;

lista_strings ::=
    LBRACE lista_cadenas:lista RBRACE
    {: RESULT = lista; :}
    | LBRACE RBRACE
    {: RESULT = experto.listaVacia(); :}
    ;

lista_cadenas ::=
    cadena:c
    {: List lista = experto.listaVacia(); lista.add(experto.procesarTexto(c)); RESULT =
lista; :}
    | lista_cadenas:lista COMMA cadena:c
    {: lista.add(experto.procesarTexto(c)); RESULT = lista; :}
    ;

lista_enteros_opt ::=
    /* vacio */ {: RESULT = experto.listaVacia(); :}
    | lista_enteros:lista {: RESULT = lista; :}
    ;

lista_enteros ::=
    NUMBER:n {: List lista = experto.listaVacia();
lista.add(Integer.valueOf(experto.toInt(n))); RESULT = lista; :}
    | lista_enteros:lista COMMA NUMBER:n {:
lista.add(Integer.valueOf(experto.toInt(n))); RESULT = lista; :}
    ;

style_opt ::=
```

```
    style_block:s
    {: RESULT = s; :}
    | /* vacio */
    {: RESULT = experto.estiloDefault(); :}
    ;

valor_simple ::=
    NUMBER:n
    {: RESULT = n; :}
    | IDENTIFIER:i
    {: RESULT = i; :}
    | BOOLEAN:b
    {: RESULT = b; :}
    | COLOR_HEX:c
    {: RESULT = c; :}
    | COLOR_RGB:c
    {: RESULT = c; :}
    | COLOR_HSL:c
    {: RESULT = c; :}
    | BORDER_STYLE:b
    {: RESULT = b; :}
    | LINE
    {: RESULT = "LINE"; :}
    | MONO
    {: RESULT = "MONO"; :}
    | SANS_SERIF
    {: RESULT = "SANS_SERIF"; :}
    | CURSIVE
    {: RESULT = "CURSIVE"; :}
    | orientacion:o
    {: RESULT = o.toString(); :}
    | cadena:c
    {: RESULT = c; :}
    | LBRACE lista_valores_opt:v RBRACE
    {: RESULT = v; :}
    ;

lista_valores_opt ::=
    lista_valores:l
    {: RESULT = l; :}
    | /* vacio */
    {: RESULT = experto.listaVacia(); :}
    ;

lista_valores ::=
    lista_valores:lista COMMA valor_simple:v
    {: lista.add(v); RESULT = lista; :}
    | valor_simple:v
```

```
    {: List lista = experto.listaVacia(); lista.add(v); RESULT = lista; :}
    ;

cadena ::=
    STRING_START cadena_partes_opt:partes STRING_END
    {: RESULT = partes; :}
    ;

cadena_partes_opt ::=
    cadena_partes_opt:acumulado cadena_parte:parte
    {: RESULT = acumulado + parte; :}
    | /* vacio */
    {: RESULT = ""; :}
    ;

cadena_parte ::=
    STRING_TEXT:t
    {: RESULT = t; :}
    | EMOJI_SPEC:e
    {: RESULT = e; :}
    ;
```